RWTH AACHEN UNIVERSITY

VEHICLE COUNTING

A detector built from scratch, and a counting pipeline for traffic video.

THE VINH NGUYEN TRONG · ALI AWADA · REXHEP AZEMI

THE TASK

Two parts, one pipeline

Detect vehicles in images, then count them in a video

01 Part 1 - build a detector by hand

Per cell of a 16 x 16 grid: one objectness score, four box values. Then NMS.

02 Part 2 - count vehicles in a video

Fine-tuned YOLO-nano, our tracker for stable IDs, one counting line.

GRADED ON: WHAT WE TRIED · WHAT WORKED · WHAT DID NOT · WHICH METHOD IS BEST, AND WHY

THE DATA

1001 images - and one video

Kaggle car-object-detection, 676 x 380 px, one vehicle class

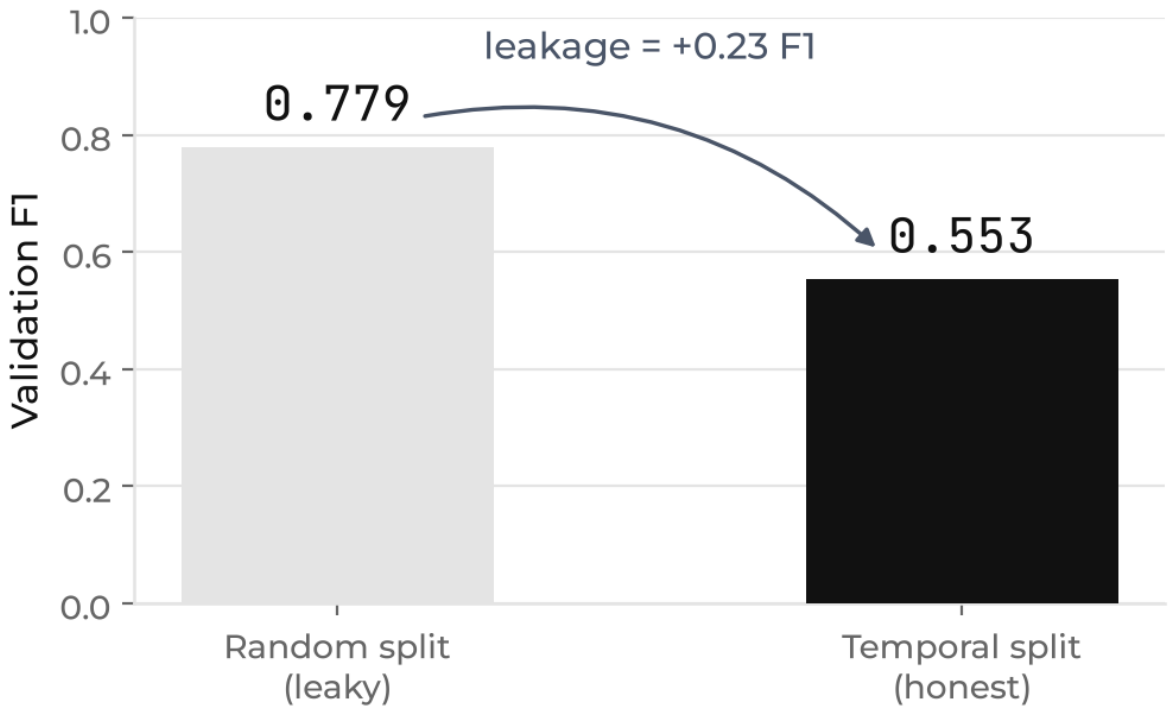
Two labelled frames, two frames of empty road - the empty ones are kept as negatives

1001	559	64.5%	1.59%	2 / 559
IMAGES	BOUNDING BOXES	FRAMES WITH NO VEHICLE	MEDIAN BOX AREA	BOXES LOST TO THE GRID

All 1001 images are frames of a single video, about 0.67 s apart - so neighbours are near-duplicates.

A random split leaks

Same model, same hyperparameters - only the split changed



WHY IT HAPPENS

A random split puts a frame in train and its twin 0.67 s later in test.

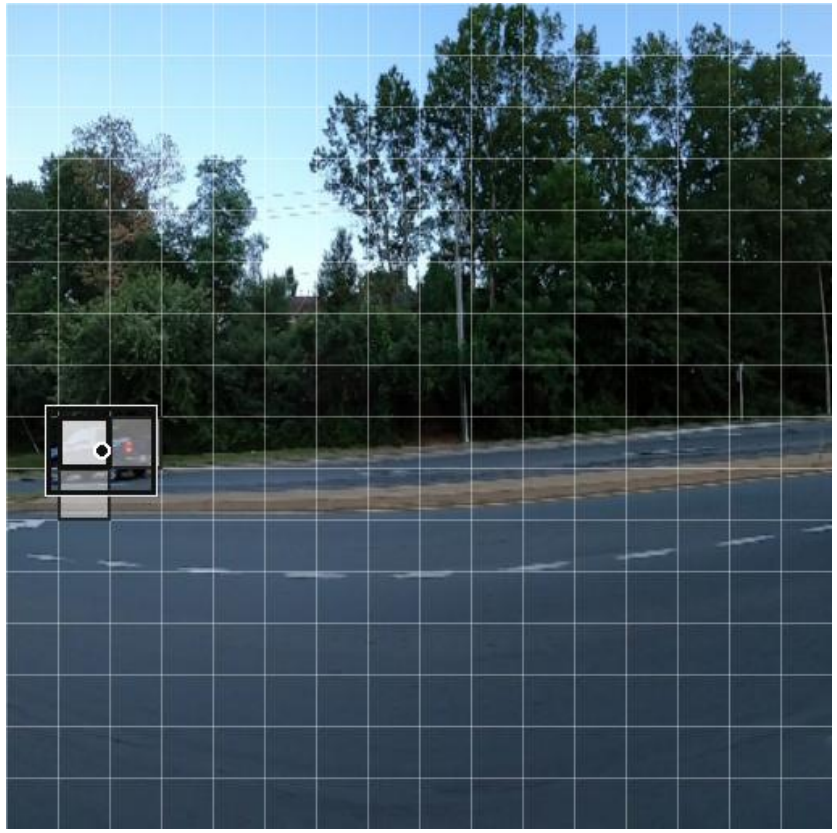
The model is scored on cars it has already memorised.

We split on time instead: 80 / 15 / 5, earliest to latest.

EVERY NUMBER FROM HERE ON IS THE TEMPORAL SPLIT

What the head predicts

One objectness score and four box values, per cell of a 16 x 16 grid



Network input 512 x 512. Bright cell holds the box centre; the pale cells are our multi-cell rule

PER CELL

`objectness`
is there a vehicle centre in this cell?

`off_x, off_y`
where in the cell the centre sits, 0 to 1

`w, h`
box size as a fraction of the image

All five values live in 0 to 1 - a plain sigmoid produces them, no anchors needed.

A unit test asserts encode - decode returns the original boxes at IoU 1.0.

STRIDE 32 · 16 x 16 GRID · 1280 NUMBERS PER IMAGE

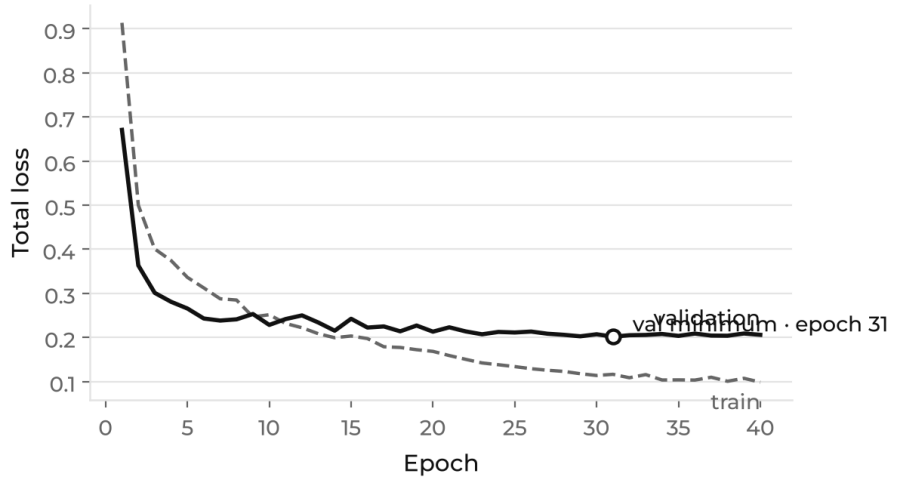
The configuration the task sheet prescribes

Frozen ResNet18, one positive cell per box, L1 box loss, 40 epochs

SPLIT	PRECISION	RECALL	F1	AP50	NOTES
Validation (150 images)	0.679	0.559	0.613	-	threshold and NMS tuned here
Test (50 images)	**0.455**	**0.395**	**0.423**	**0.213**	**38 boxes only**

The test split is too small to carry the argument alone.

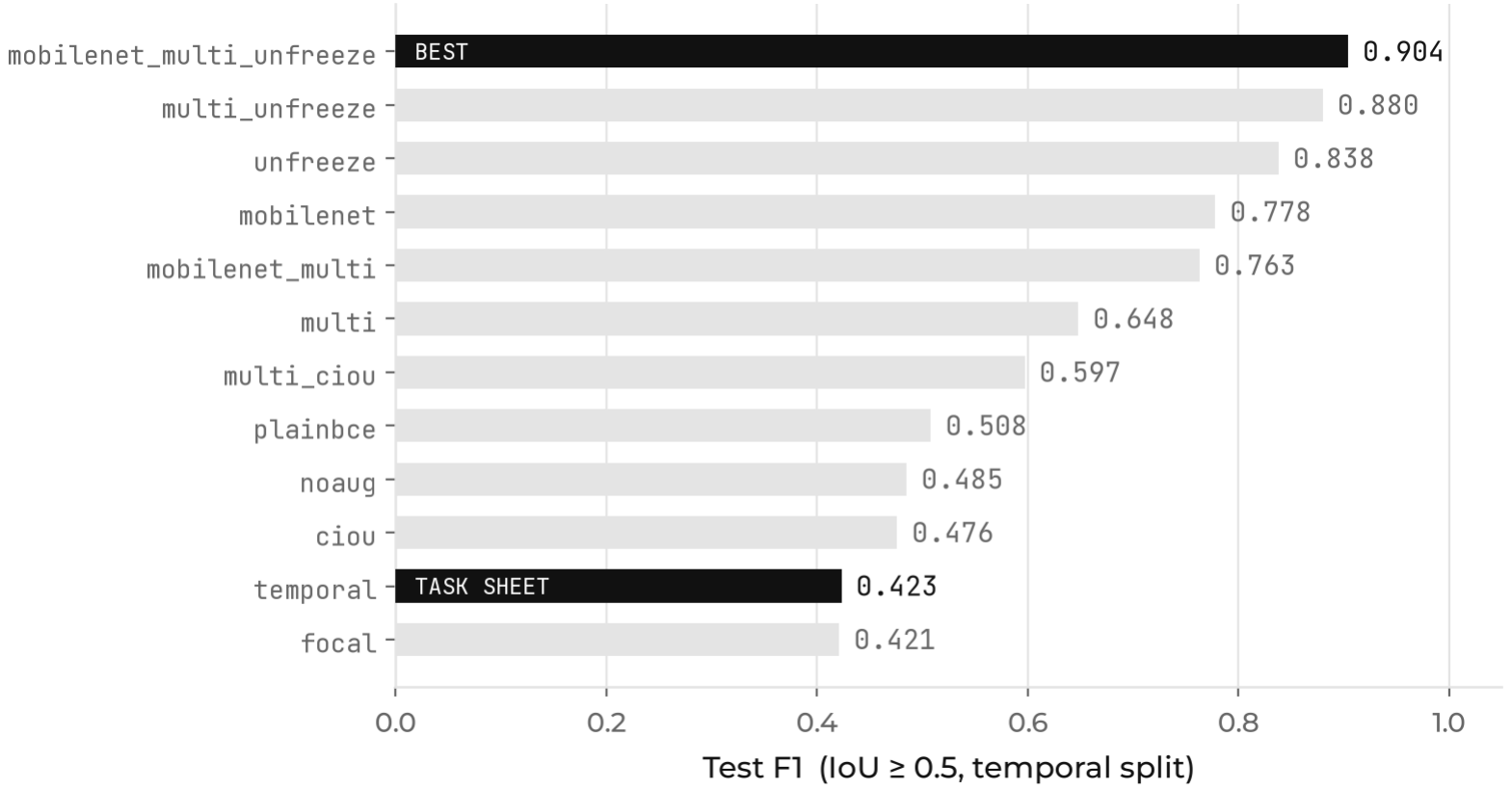
50 images, 38 boxes - one bad frame moves F1 by several points. The 80/15/5 ratio is the task sheet's, so validation is our primary number.



TRAINING STOPS HELPING AT EPOCH 17 - MORE DATA, NOT MORE EPOCHS

Twelve configurations, one change at a time

Test F1 at IoU 0.5, temporal split, threshold and NMS tuned per run



THE GAP

0.423

TASK-SHEET BASELINE

0.904

OUR BEST RUN

AP50 goes 0.213 to 0.871 - four times. Two changes account for almost all of it.

Two changes, almost all of the gain

Each was measured on its own before they were combined

LEVER 1 · UNFREEZE THE BACKBONE + 0.42 F1

The frozen features were the bottleneck, not the head.

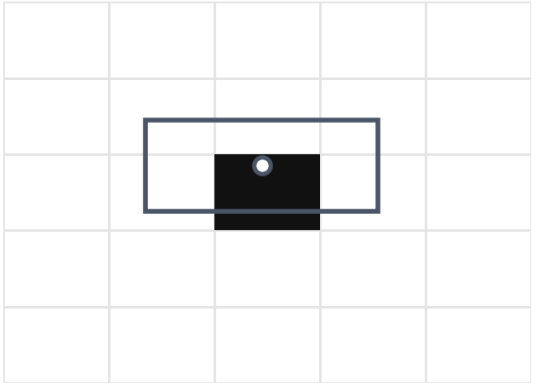
ImageNet is object-centric photography; our vehicles are small, blurred and shot from a dashcam.

LEVER 2 · MULTI-CELL ASSIGNMENT + 0.23 F1

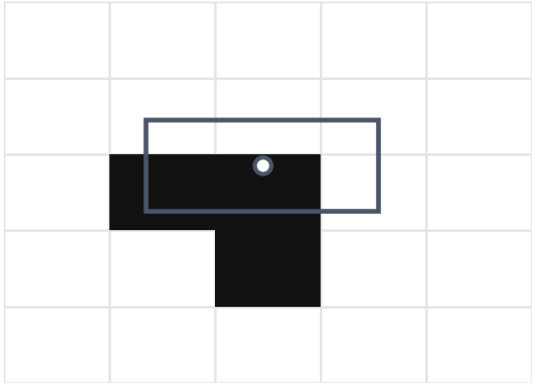
One positive cell per box is 453 signals in the whole training set.

Neighbouring cells fire anyway but were never taught which box to emit, so they emit fragments.

Task sheet: 1 positive cell



Ours: 3 positive cells



Train the centre cell plus its two nearest neighbours on the same box.

They become agreeing votes that NMS merges. False positives on test: 18 to 10.

The negative results, kept on purpose

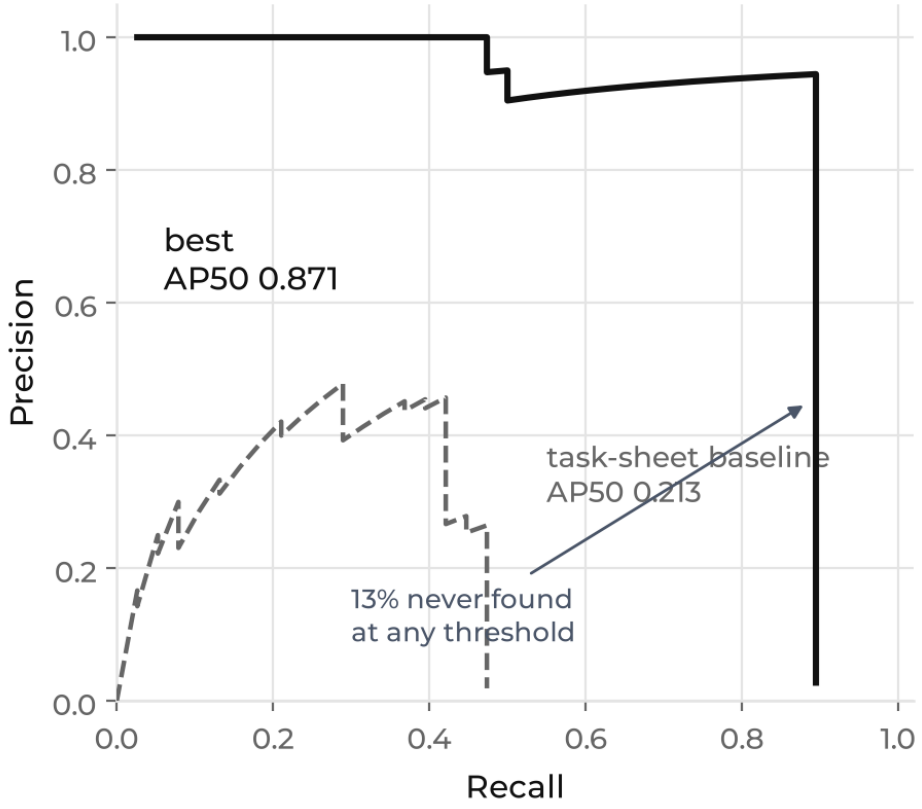
Each was a reasonable idea before it was measured

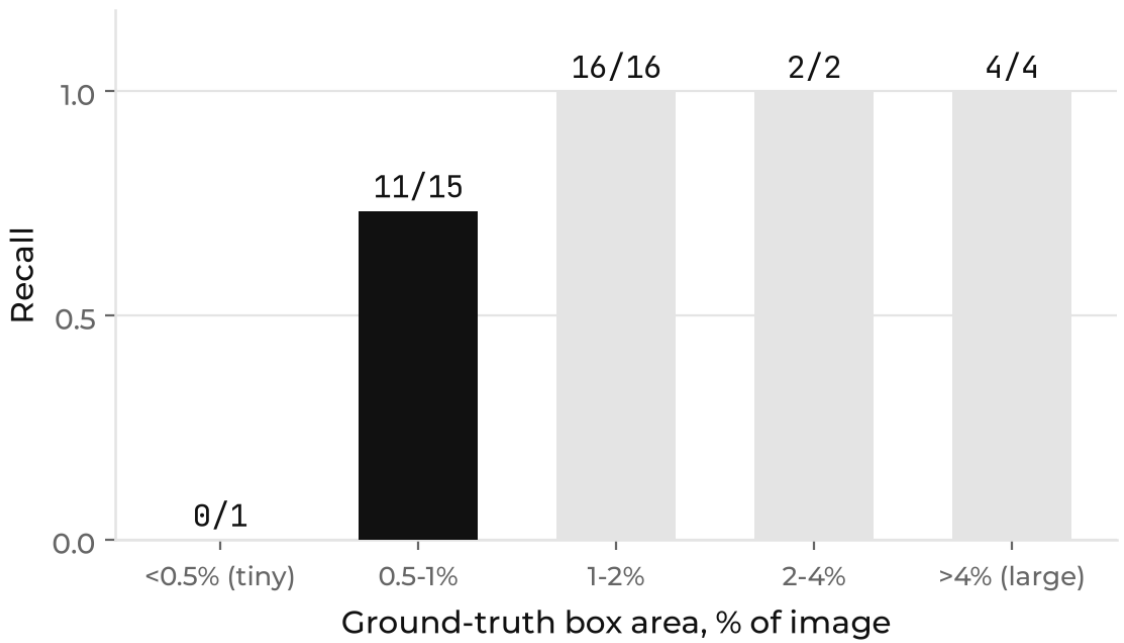
ATTEMPT	RESULT	WHY IT FAILED
CIoU box loss	0.476 / 0.597	unstable gradients on tiny targets; L1 is the easier optimisation
Focal loss	0.421	built for dense anchor detectors; it starved the head
Plain BCE	0.508	collapsed to 'no vehicle' - 99% cell accuracy, useless detector
Flip + colour jitter	0.485 vs 0.423	the data lacks scale and viewpoint, not colour
Larger input, 640 / 768 px	0.917 / 0.889	finds the same 33 vehicles - one false positive fewer
Finer grid, stride 16	0.822	resolution bought, semantic depth paid - that is what an FPN solves

WE GOT THIS TABLE WRONG TWICE OURSELVES: A CHECKPOINT READ MID-TRAINING, BOX AREA IN INPUT PIXELS

Every miss is a small, distant vehicle

Precision holds above 0.95 out to recall 0.87, then the curve falls off a cliff





Above 1% of image area, recall is 22 / 22.

The fix is a feature pyramid or more data - not more pixels.

PART 1 · RESULT

Test precision 0.943, recall 0.868

MobileNetV3, multi-cell assignment, backbone fine-tuned - 512 px as specified

0.943	0.868	0.904	0.871	33 / 2 / 5
PRECISION	RECALL	F1	AP50	TP / FP / FN

Test frames. Grey = ground truth, black = prediction with its confidence

Precision is a lower bound - some frames leave visible cars unboxed.

Detect, track, count

Three stages - the count depends on all three, not just the detector

FRAME
1798 frames, 60 s

→

YOLO11n
fine-tuned, one class

→

TRACKER
IoU + Hungarian, ours

→

LINE COUNTER
centre crosses, once per ID

WHAT YOU SEE

Green box + #ID
a confirmed track

Orange box
already counted

Red line
the counting line, y = 0.65

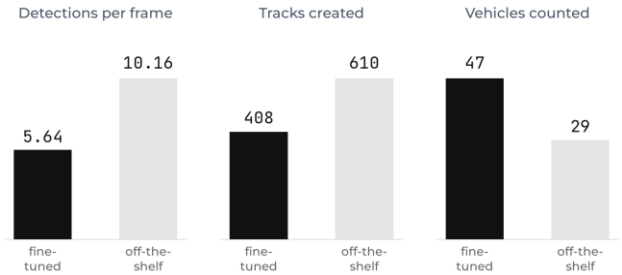
Panel
running count per direction

Counted when the centre crosses - not when it appears.

Twice the detections, fewer vehicles counted

Same frame, same tracker, same line - only the detector changes

A typical frame: the off-the-shelf model adds parked cars far from the line



Fine-tuning helped - we predicted it would hurt.

Unstable boxes fragment tracks: 21.0 tracks per counted vehicle against 8.7.

IoU association, written from scratch

Hungarian against a greedy baseline, both unit-tested on synthetic boxes

---

01 Associate detections to tracks by IoU, above 0.3

Hungarian is optimal over the whole frame; greedy takes the best pair first.

02 Confirm before counting, forget after 0.33 s unseen

Thresholds in seconds, scaled by the video's own frame rate.

03 Count when the centre crosses, once per ID

A proper segment intersection, so the line's infinite extension does not count.

Every tracked path in the clip - 408 tracks for 47 counted vehicles

A bug the unit tests caught, and the video never would have.

A centre landing exactly on the line was a third state, so the crossing vanished. On screen that looks like nothing at all - just a count that is too low.

Choosing the video, not writing the tracker

The binding requirement is invisible in a still frame

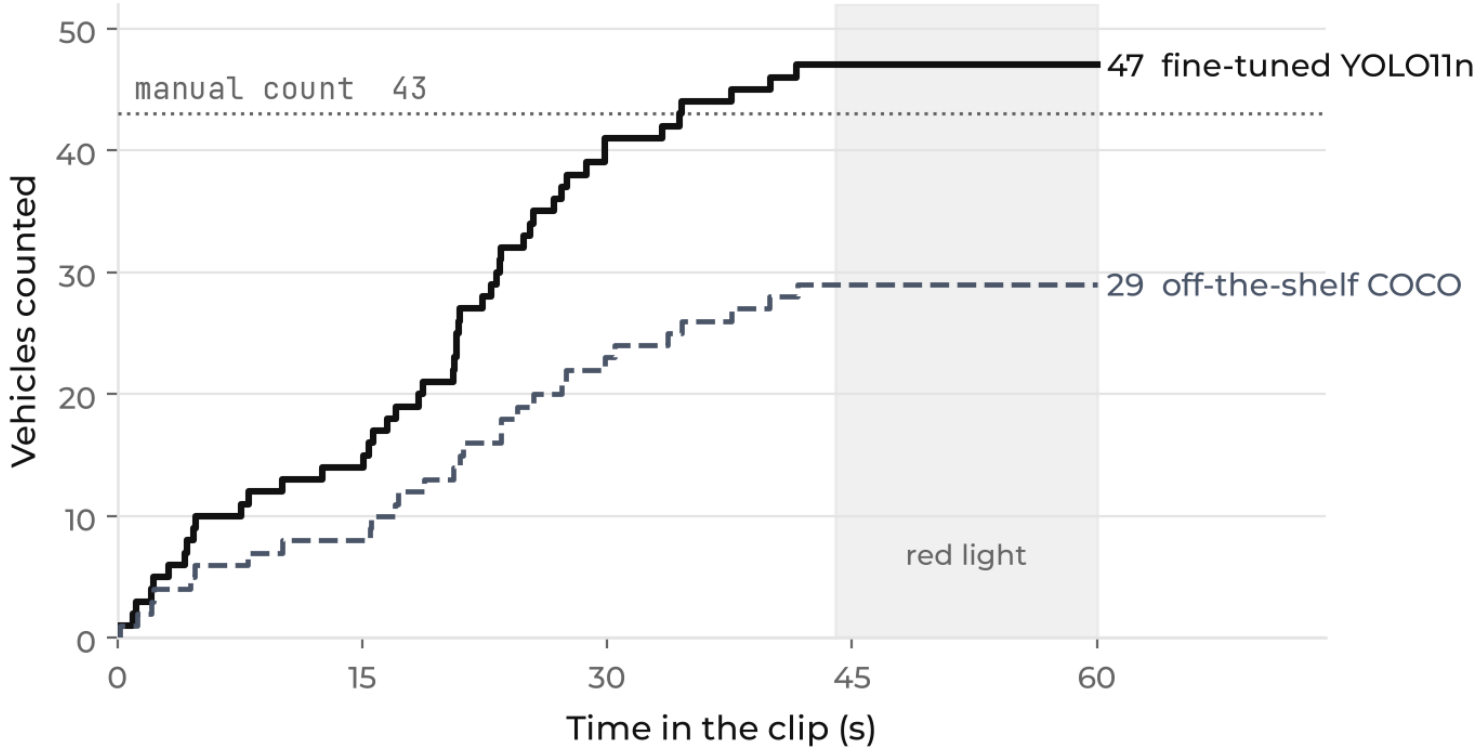
CANDIDATE	DET/FRAME	MOVING TRACKS	BEST LINE	VERDICT
Dual carriageway, dusk	18.7	8	3 crossings	too dense to hand-count
US freeway	16.2	-	-	licence forbids redistribution
Daylight, free-flowing	7.4	22	12 / 0	one direction only
T-junction	9.2	34	15 (14 one way)	looked perfect - traffic disperses
Intersection (chosen)	**8.8**	**93**	**42 = 31 / 11**	**one line, both flows**

"Both directions are visible" is not the requirement.

One line has to exist that both flows cross - and that is only visible in the trajectories.

47 counted against a manual 43

Same 60 s, same counting rule for the human and the machine



RESULT

+9.3%

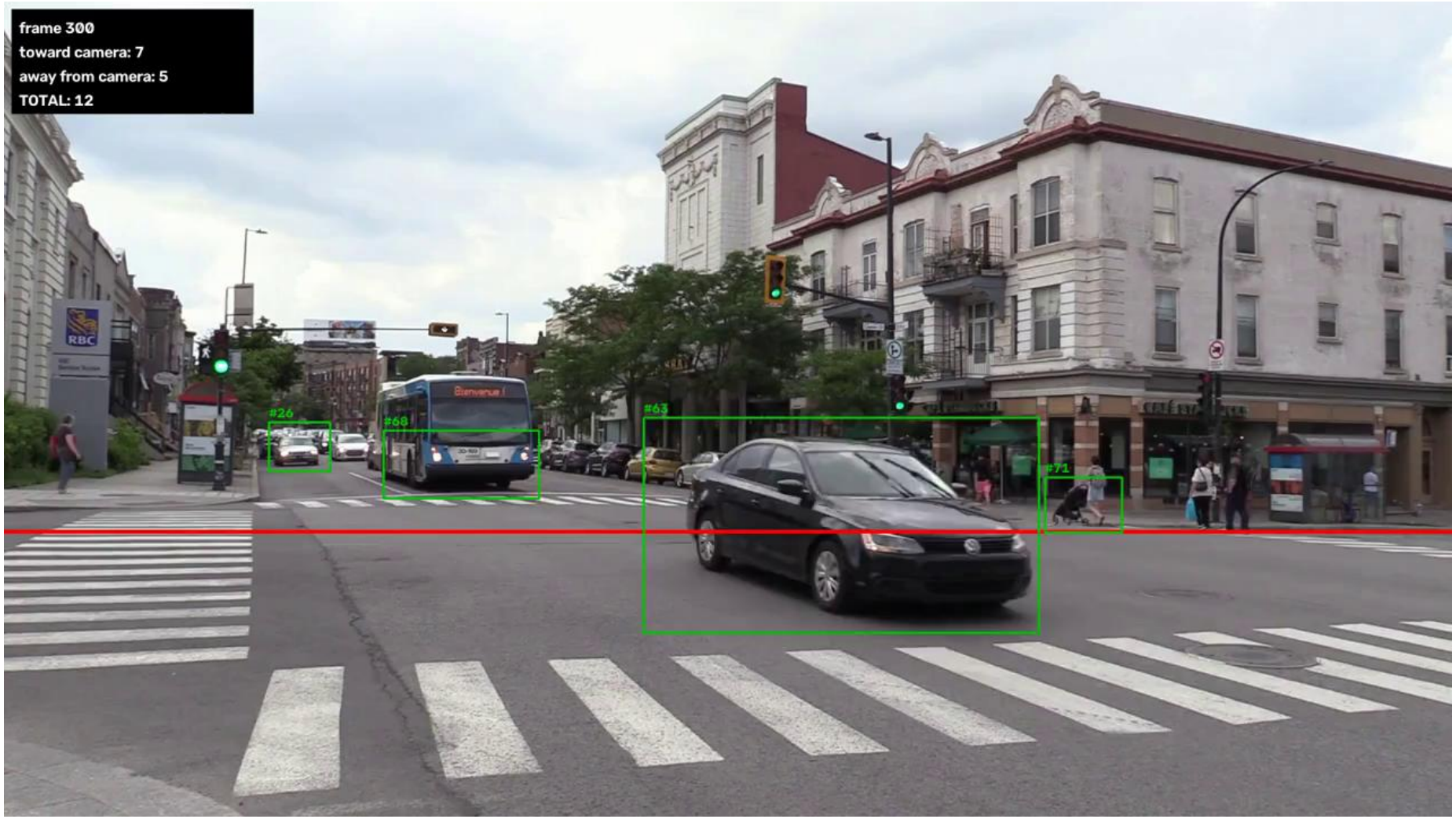
NET OVER-COUNT, 4 VEHICLES

Hungarian and greedy tie: 47 and 47, 29 and 29.

Without a Kalman filter a dropped frame ends a track, the vehicle returns with a new ID and crosses twice - so the error goes up, as theory says.

STILL OPEN: THE MANUAL COUNT HAS A TOTAL, NOT A PER-DIRECTION SPLIT

CLICK TO PLAY



30 SECONDS OF THE COUNTED OUTPUT · BOXES, TRACK IDS, THE LINE AND THE RUNNING COUNT

9.3% is where a street-level camera lands

Published counting errors, split by camera height

SYSTEM	CAMERA	COUNT ERROR	SOURCE
YOLO + visual rhythm	top view	0.85-1.4%	Ribeiro 2025
YOLOv5 + DeepSORT	CCTV, 15 m up	1.9%	Tashkent 2023
YOLOv3 + SORT, 6 intersections	14-40 m	5.5%	Khazukov 2020
Ours: YOLO11n + IoU/Hungarian, no Kalman	**street level**	**9.3%**	**this project**
Best method at low vantage	under 5 m	9.9-11.0%	Pakdamansavoji 2025
IoU tracker without Kalman, same ablation	DOT cameras	over 37%	Mandal 2021

Camera height explains more than the algorithm does.

The same footage scores 1.81% top-down against 28.58% in perspective view. On our clip 43.5% of detections overlap another one.

What we would fix first

Measured limits, not a wish list

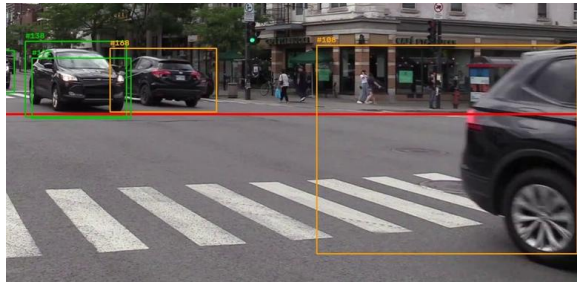
KNOWN LIMITS

OCCLUSION
43.5% of detections overlap another one

OVERSIZED BOXES
a box up to 22.7% of the frame - its centre crosses too early

NET HIDES GROSS
+4 could be 6 double counts and 2 misses

ONE CLIP
no confidence interval; one vehicle is already 2.3%



One box spanning a vehicle and the background - its centre crosses the line at the wrong moment

NEXT, IN ORDER OF EXPECTED PAYOFF

01 A Kalman filter, or DeepSORT
the published ablation of our exact design says the error lives here

02 A feature pyramid for Part 1
every remaining miss is a vehicle under 1% of the image

03 Scale and viewpoint augmentation
flip and jitter changed nothing; the data lacks scale

04 Finish the per-direction manual count
the only external ground truth the project has

THANK YOU · QUESTIONS

IT RUNS ON DATA WE HAVE NEVER SEEN

Part 1, any folder of images

```
python -m src.part1.predict --images <dir> [--csv ground_truth.csv]
```

Part 2, any video

```
python -m src.part2.suggest_line --video new.mp4   then   run_count --video new.mp4
```

Everything reproducible

one config file, 33 unit tests, weights and the output video ship with the code